

Ejecucion del split de training parcial

Contexto

En pasadas pruebas con la implementacion del eje D se ejecuto el split de training completo usando el script @attachment:run_training_full.sh . En estas pruebas se resolvieron 140 tareas de 400.

Tras la implementacion del eje B, ahora se realizan 4 veces mas de pasos. Para no volver a ejecutar tantos pasos, la idea es que se ejecuten solamente las 260 tareas no superadas en el ultimo recorrido del split.

Aqui puedes ver los 140 tareas resueltas #attachment:all_results_axis_D.txt en un documento compactado.

Objetivo

- Realiza un analisis y adapta el script @attachment:run_training_full.sh para realizar la ejecucion del split despues de todos los cambios realizados en el eje de mejora B. Adicionalmente añade un argumento opcional para este caso en concreto en el que queremos ejecutar el split particular de las 260 tareas que fallaron anteriormente.

Consideraciones

- Tras algunas apreciaciones visuales, simplemente para que tengas en consideracion que se han modificado los parametros de AX_WORKERS, EM_PER_WORKER_GB y MEM_RESERVE_GB en @attachment:run_training_full.sh para que se ejecuten de manera simultanea mas tareas. Observando que habia espacio en la memoria RAM y la targeta grafica libre. No tienes que hacer nada al respecto, simplemente tenerlo en cuenta

Updated todo list

Voy a trazar primero cómo `run_training_full.sh` delega en `run_full_split.sh`, qué argumentos/env admite hoy y cómo filtra tareas `parallel_train.py`. Después diseñaré un modo opcional que calcule las 260 pendientes desde la lista de 140 resueltas, sin hardcodear silenciosamente otro split ni interferir con los valores de concurrencia que ya ajustaste.

Plan: Training B sobre 260 pendientes

Se adaptará `run_training_full.sh` para ejecutar Eje B por defecto y aceptar:

```
.run_training_full.sh --axis-d-unsolved
```

Este modo excluirá los 140 IDs de `all_results_axis_D.txt`, validará que quedan exactamente 260 tareas y no sobrescribirá los resultados anteriores.

Cambios principales

- Crear `task_selection.py` para parsear y validar la lista 140→260 conservando el orden original del dataset.
- Activar desde el wrapper:
 - `--accel-preset compile`
 - `--eje-b`
 - seeds `0,1,2,3`
 - 2000 iteraciones
 - stride 4
- Mantener tus valores actuales:
 - `MAX_WORKERS=8`
 - `MEM_PER_WORKER_GB=11`
 - `MEM_RESERVE_GB=14`
- Ejecutar mediante `--task-ids` solamente las 260 pendientes.
- Aislar los artefactos parciales en:
 - Outputs: `split_results/training/eje_b_axis_d_unsolved/`
 - Estado: `.partial_eje_b/`
- Corregir la recuperación para que `--state-dir` también controle `.task_recovery.json`. Esto evita que el estado eager de `fc5c309` heredado de Eje D bloquee Eje B.
- Mantener reanudación por `(task, seed)`: relanzar el mismo comando ejecutará solamente seeds pendientes.
- La salida contendrá deliberadamente 260 tareas; no se generará una combinación artificial de 400.

Carga esperada

$$260 \text{ tareas} \times 4 \text{ seeds} \times 2000 = 2\,080\,000 \text{ pasos}$$

Validación

- Parser estricto: 140 IDs válidos, únicos y conocidos; 260 pendientes.
- Tests de wrapper, recuperación aislada, tracking y rutas de salida.
- Suite completa de tests, `bash -n` y `git diff --check`.
- No se iniciará automáticamente la campaña larga.